AI ENGINEERING DEMYSTIFIED / REPO 04

LangGraph fundamentals

INTERVIEW CHEATSHEET 24 code patterns / 19 notebooks / 12 modules

STATE → NODE → UPDATE → NEXT

Read down each column. Module numbers appear on every card.

CORE IMPORTS

```
from typing import Annotated, Literal, TypedDict
from operator import add
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.types import Command, interrupt, RetryPolicy
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver
```

STATE

What moves through the graph

01 State + reducers — M01/12

Plain keys overwrite; reducers merge updates, so return only the new delta.

```
class State(TypedDict):
    n: int
    events: Annotated[list[str], add]
def step(s):
    return {"n": s["n"] + 1,
            "events": ["step"]}
```

02 Build, compile, invoke — M01

Nodes return partial updates; edges order work; compile creates the runnable.

```
g = StateGraph(State)
g.add_node("step", step)
g.add_edge(START, "step")
g.add_edge("step", END)
app = g.compile()
app.invoke({"n": 0, "events": []})
```

03 MessagesState + LLM node — M01/02

add_messages appends new IDs and replaces matching IDs; return the new reply.

```
class Chat(MessagesState):
    topic: str
def chat(s):
    reply = llm.invoke(s["messages"])
    return {"messages": [reply]}
g = StateGraph(Chat)
```

04 Choose a state schema — M03

TypedDict is hints; dataclass adds defaults; Pydantic validates node inputs.

```
from dataclasses import dataclass
from pydantic import BaseModel
@dataclass
class Defaults: n: int = 0
class Validated(BaseModel):
    n: int = 0
```

05 Input / internal / output — M07

Separate the public I/O contract from internal channels; schemas are not access control.

```
class Input(TypedDict): question: str
class Output(TypedDict): answer: str
class Internal(Input, Output): calls: int
g = StateGraph(Internal,
    input_schema=Input,
    output_schema=Output)
```

06 Prune model context — M12

RemoveMessage deletes by ID with add_messages; preserve valid tool-call pairs.

```
from langchain_core.messages import RemoveMessage
def trim(s):
    old = s["messages"][:-2]
    return {"messages": [
        RemoveMessage(id=m.id) for m in old
    ]}
```

FLOW

Where execution goes next

07 Conditional edges — M02

A router reads state and selects a registered destination without updating state.

```
def route(s):
    return "yes" if s["n"] > 0 else "no"
g.add_conditional_edges("check", route,
    {"yes": "accept", "no": "reject"})
```

08 Command = update + route — M04

Command changes state and chooses the next node; static outgoing edges still run.

```
def decide(s) -> Command[Literal["done"]]:
    return Command(
        update={"n": s["n"] + 1},
        goto="done")
```

09 Guard a cycle — M11

Loop until success or an explicit cap; recursion_limit counts supersteps, not node calls.

```
def again(s):
    stop = s["ok"] or s["n"] >= 3
    return END if stop else "work"
g.add_conditional_edges("check", again)
g.add_edge("work", "check")
app.invoke(inputs, {"recursion_limit": 20})
```

10 Subgraph: shared keys — M08

A compiled child can be a parent node when their state channels overlap.

```
child = child_builder.compile()
g.add_node("child", child)
g.add_edge(START, "child")
g.add_edge("child", END)
```

11 Subgraph: map schemas — M08

Wrap the child invocation to translate the parent input and child output.

```
def adapt(s):
    out = child.invoke({"text": s["task"]})
    return {"result": out["answer"]}
g.add_node("child_adapter", adapt)
```

12 Tool-call loop — M06/08/09

Bind tools to the model; route AI tool calls to ToolNode, then return to the LLM.

```
from langgraph.prebuilt import (
    ToolNode, tools_condition)
llm = llm.bind_tools(tools)
g.add_node("tools", ToolNode(tools))
g.add_conditional_edges("assistant", tools_condition)
g.add_edge("tools", "assistant")
```

RUNTIME

How each run executes

13 Per-run configuration — M04/07

RunnableConfig carries thread identity, tracing settings, and configurable run options.

```
from langchain_core.runnables import RunnableConfig
def node(s, config: RunnableConfig):
    lang = config["configurable"]["language"]
    return {"answer": lang}
app.invoke(inputs, {"configurable": {
    "thread_id": "t1", "language": "Hindi"}})
```

14 Runtime context — M04

Inject read-only run dependencies through context; graph updates belong in state.

```
from langgraph.runtime import Runtime
class Ctx(TypedDict): user_id: str
def node(s, runtime: Runtime[Ctx]):
    return {"owner": runtime.context["user_id"]}
g = StateGraph(S, context_schema=Ctx)
app.invoke(inputs, context={"user_id": "u1"})
```

15 Async node + ainvoke — M09/12

Await I/O in async nodes and use ainvoke for the completed graph result.

```
async def chat(s):
    reply = await llm.ainvoke(s["messages"])
    return {"messages": [reply]}
g.add_node("assistant", chat)
result = await app.ainvoke(inputs, cfg)
```

16 Stream updates / values / tokens — M02/09

updates emits node deltas; values emits full state; messages emits LLM chunks + metadata.

```
for state in app.stream(inputs, cfg,
    stream_mode="values"): print(state)
async for delta in app.astream(inputs, cfg,
    stream_mode="updates"): print(delta)
async for chunk, meta in app.astream(inputs, cfg,
    stream_mode="messages"): print(chunk.content)
```

17 Retry transient failures — M10/12

Retry only matching errors with backoff; max_attempts includes the first attempt.

```
g.add_node("fetch", fetch,
    retry_policy=RetryPolicy(
        max_attempts=3, initial_interval=1,
        backoff_factor=2, max_interval=10,
        jitter=True, retry_on=ConnectionError))
```

18 Parallel fan-out + merge — M12

Same-step writes to one plain key conflict; use a suitable reducer and explicit join.

```
class Totals(TypedDict):
    score: Annotated[int, add]
g.add_edge(START, "a")
g.add_edge(START, "b")
g.add_edge(["a", "b"], "join")
```

PERSISTENCE

How work pauses and returns

19 Checkpoint + thread identity — M05/12

Reuse thread_id to restore progress; RAM is for tests, SQLite persists to disk.

```
app = g.compile(checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "t1"}}
app.invoke(inputs, cfg)
with SqliteSaver.from_conn_string("ck.db") as db:
    app = g.compile(checkpointer=db)
    app.invoke(inputs, cfg)
```

20 Inspect, replay, fork — M05/12

Inspect values/next/tasks; replay reruns later work; update_state applies reducers and forks.

```
snap = app.get_state(cfg)
hist = list(app.get_state_history(cfg))
prior = next(cp for cp in hist if cp.next)
app.invoke(None, prior.config)
fork = app.update_state(prior.config, {"n": 99})
app.invoke(None, fork)
```

21 Interrupt: approve / reject — M06

A dynamic pause needs a saver and thread; resume restarts the node from its beginning.

```
def review(s):
    decision = interrupt("Approve?")
    return Command(goto="yes" if
        decision == "approve" else "no")
app.invoke(Command(resume="approve"), cfg)
```

22 Interrupt: edit / validate — M06

Resume supplies the interrupt return value; validate it before returning an update.

```
def edit(s):
    text = interrupt({"draft": s["draft"]})
    while not isinstance(text, str):
        text = interrupt("Enter text")
    return {"draft": text}
app.invoke(Command(resume="new draft"), cfg)
```

23 Breakpoint / crash recovery — M06/12

Static pauses and repaired failures continue with None; dynamic pauses use Command(resume=...).

```
app = g.compile(
    checkpointer=InMemorySaver(),
    interrupt_before=["review"])
app.invoke(inputs, cfg)
app.update_state(cfg, {"n": 7})
app.invoke(None, cfg)
```

24 Idempotent effects — M12

Retries can repeat effects; use durable atomic deduplication or a provider idempotency key.

```
ledger = {}  # Single-process sketch only
def pay_once(key):
    if key not in ledger:
        ledger[key] = provider_pay(key)
    return ledger[key]
```

Interview recall: state stores data | edges choose work | reducers merge writes | checkpoints save progress | interrupts pause durably.

Watch for: returning full history to an append reducer; effects before interrupt(); expecting checkpoints to guarantee exactly-once effects.

Independent patterns: g = builder; app = compiled graph. Supply named nodes, schemas, llm/tools, and inputs; cfg carries thread_id.

Source: modules 01-12 notebooks | LangGraph 1.2.12 | A3 landscape